

EV Battery Thermal Twin

Physics-Informed Neural
Network (PINN) vs. Data-
Driven MLP

**Technical Review & Pipeline
Analysis**

Prepared by Antigravity AI
Codebase Specialist

1. The Problem We Are Solving

The Target System

For electric vehicle safety and longevity, the Battery Management System (BMS) must estimate **cell core temperatures in real-time**.

- High temperatures trigger safety cutoffs.
- Accurate temperature tracking allows faster charging profiles.
- Requires tracking dynamic thermal behavior during driving cycles

The Technical Challenge

We are caught between two unsuitable options:

1. **Finite Element Analysis (FEA):** Physically accurate but requires minutes/hours of computation—impossible for real-time edge devices.
2. **Simplified Equations:** Fast but inaccurate under rapid transient

2. What Has Been Done Before

Approach A: Pure Data NNs

Neural networks trained strictly on data signals (Voltage, Current, SOC) to predict temperature changes.

✓ **High Flexibility:**

Fits highly non-linear dynamics easily without physical parameters.

✗ **Unbounded Extrapolation:**

Violates conservation laws and drifts

Approach B: Simplified RC Models

Resistance-Capacitance thermal equivalent circuits representing heat transfer via ODEs.

✓ **Physical Consistency:**

Guarantees thermal bounds and respects energy conservation laws.

✗ **Calibration Bottleneck:**

Extremely tedious to calibrate and

3. What Has NOT Been Done (The Gap)

The Multi-Step Rollout Gap

Previous ML models are evaluated strictly on **single-step-ahead prediction** ($T_t \rightarrow T_{t+1}$).

Because the step size is tiny ($\Delta t = 0.01$ s), temperature changes very little. Models exploit this

Physical Prior Mismatches

Prior attempts at regularizing networks with physics equations hardcode generalized parameters (like textbook mass and heat capacities).

This creates a direct conflict with the empirical data generation environment, causing the optimizer

4. The Idea Behind the Solution

A Physics-Informed Neural Network (PINN) with Trainable Parameters

Our solution couples a deep neural network's representative power with the governing lumped-parameter thermal differential equation directly inside the optimizer loss graph.

1. Physics-Guided Loss Regularizer

Instead of hoping the network

2. Dynamic Parameter Inference

Rather than hardcoding the heat

5. Step-by-Step Implementation Timeline

We solved this challenge systematically across five distinct development phases:



Let's walk through the technical and engineering details of each step.

Step 1: Data Sweep & Split Isolation

To train the models, we executed a parameterized grid simulation in MATLAB/Simulink:

The Grid Configurations

- **Model:** `Ebike_Thermal_DT_v1.slx` (electrothermal)
- **Drive Cycles:** FTP75, US06, UDDS, HUDDS, WLTP1, WLTP2
- **Initial Temperatures:** 0°C, 10°C, 25°C, 40°C, 50°C
- **Mass Scales:** 0.8, 1.0, 1.2 (affects thermal inertia)

Preventing Data Leakage

Standard random splits leak temporal correlations, inflating model accuracy.

Decision: We reserved the entire `cycleWLTP2` drive cycle exclusively as the **TEST SET**. The model has never seen this dynamic drive pattern

Step 2: Stream Preprocessing

To handle large-scale dataset loading without crashing RAM, we designed a custom stream pipeline:

1. Incremental Preprocessing

We designed standardizing scalers to fit incrementally via `joblib`, avoiding loading the entire dataset into memory simultaneously.

Saved features: `scaler_x.joblib`,
`scaler_y.joblib`.

2. Sequential Batch Sampler

Global shuffling destroys time-series continuity, preventing correct calculations of $\frac{dT}{dt}$.

Our custom sampler ensures that each batch only draws indices from a **single simulation file**

Step 3: Dual-Loss Model Formulations

Baseline MLP

Standard Feedforward network mapping 7 features to temperature delta (ΔT).

```
self.network = nn.Sequential(  
    nn.Linear(7, 64), nn.ReLU(),  
    nn.Linear(64, 64), nn.ReLU(),  
    nn.Linear(64, 32), nn.ReLU(),  
    nn.Linear(32, 1)  
)
```

Physics-Informed NN

Identical architecture, but convective parameter hA is trainable. We map it via `Softplus` to avoid vanishing gradients during optimization:

```
# Trainable parameter initialized at 0.1  
self.hA_raw = nn.Parameter(  
    torch.tensor(raw_init)  
)  
def get_hA(self):
```

Step 3: PINN Governing Equation Residuals

The PINN checks physical validity at every epoch using the lumped capacity equation:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{physics}}$$

Governing Physical Law

$$\frac{dT}{dt} = \frac{P_{\text{loss}}}{m C_p} - hA(T - T_{\text{amb}})$$

Residual Loss Computation

```
# Inside forward / loss pass:
dTdt_pred = Delta_T_pred_phys / dt
hA = self.get_hA()
Q_loss = hA * (T_t - AmbientTemp)
```

Step 4: Diagnosing Physical Conflicts

During testing, the PINN failed to converge and hA clamped to its minimum. We audited the dataset parameters:

1. The mC_p Mismatch

The code assumed a thermal capacity of $m C_p = 4000$ J/K.

We ran a linear regression directly on the Simulink data stream and found the actual capacity was ****900 J/K**** (4.4x lower). Enforcing a prior that was so far off created

2. Casing-Core Phase Lag

Simulink utilized a **two-node** core-surface model. When current drops to 0, casing temperature continues to rise as heat transfers outward from the hotter core.

A **single-node** equation cannot model this lag, requiring a negative

Step 5: Hardware & Performance Optimizations

To ensure rapid iterations on local developer hardware (NVIDIA RTX 4060 Laptop GPU), we optimized performance:

Memory & Sync Bottlenecks

- **Non-Blocking Transfers:**

Configured

```
.to(device,  
non_blocking=True)
```

to allow GPU computations and memory copies to

Tensor Core Acceleration

- **AMP Autocast:** Enabled float16 Automatic Mixed Precision (AMP) to maximize Tensor Core utilization.
- **Precision Selection:** Used

```
torch.set_float32_matmul_precision("high")
```

 to enable TF32 support.

Summary of Results: The Exposure Bias Divergence

We evaluated both models on a full **1500-second autoregressive rollout** on the unseen **WLTP2** test set:

Evaluation Metric	Baseline MLP	Physics-Informed NN (PINN)
Single-Step Test MAE	0.00002 K	0.00003 K
Rollout MAE (compounded)	3.85 K	4.70 K
Rollout Max Error	8.56 K	6.46 K

Compounding Error

During rollout, the MLP's prediction errors accumulate, causing steady

Physical Bound

The PINN's physical loss constraints limit peak error drift, but are bound by

Roadmap: Next Stabilization Phases

To resolve the remaining gaps and deliver a highly accurate Digital Twin, we propose the following roadmap:

Phase 1: Rollout-Based Loss

Train the model using **Truncated Backpropagation Through Time (BPTT)** over 50-step prediction windows instead of 1-step targets. This directly minimizes exposure bias.

Phase 2: Two-State PINN

Upgrade the governing equations to represent both the Core temperature (T_c) and Casing temperature (T_s):

$$\frac{dT_c}{dt} = \frac{P_{\text{loss}} - R_{\text{cs}}(T_c - T_s)}{C_c}$$

$$\frac{dT_s}{dt} = \frac{R_{\text{cs}}(T_c - T_s)}{C_s}$$